

ARQUITETURA · TECNOLOGIAS

Sistema de Moeda Estudantil

Arquitetura e Tecnologias

01 · VISÃO DO PRODUTO Perfis e valor

Reconhecer mérito transforma desempenho em oportunidade

Uma moeda virtual conecta reconhecimento, benefícios e parceiros.

Aluno

Recebe moedas e resgata benefícios.

Professor

Reconhece alunos e envia moedas.

Empresa parceira

Oferece vantagens e valida cupons.

02 · ARQUITETURA Separação de responsabilidades

A arquitetura separa interface, regras e persistência

Cada camada tem uma função clara no fluxo principal.



React

View

Controller REST

Entrada e saída JSON



Service

Regras de negócio



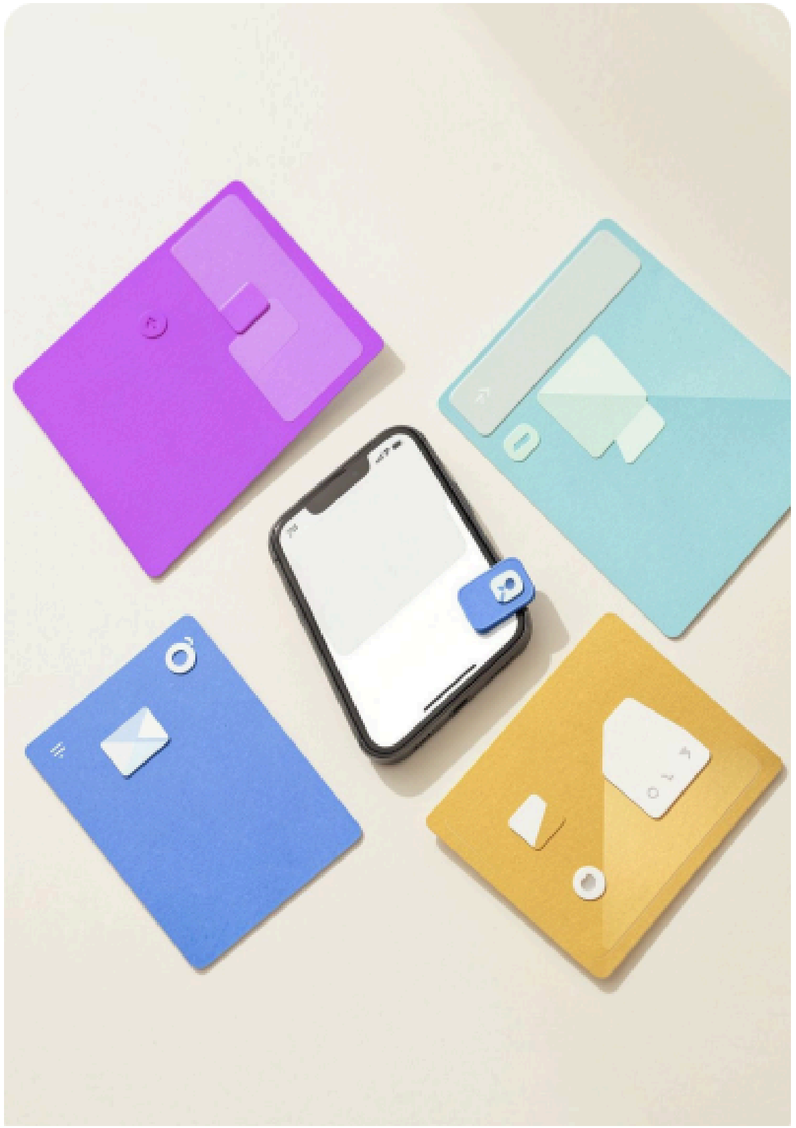
DAO

JPA/Hibernate



Banco H2

Armazenamento



React concentra a experiência de cada perfil

Uma SPA em React 18 organiza a navegação e adapta a experiência de aluno, professor e empresa.

Estrutura

pages/ reúne login, cadastros e telas de cada perfil.

Composição

components/ concentra layout, rotas protegidas, formulários e modais.

Sessão

AuthContext.jsx mantém o usuário autenticado.

Integração

api.js concentra as chamadas REST. React Router e Vite completam a base.

04 · API Entrada controlada

Controllers expõem uma API segura e desacoplada

1

Receber

Requisições HTTP chegam ao controller.

2

Validar

Entradas são verificadas antes do processamento.

3

Traduzir

DTOs como records Java protegem o modelo interno.

4

Responder

O cliente recebe JSON, sem depender do esquema do banco.

A API protege entidades e senha ao manter o contrato externo separado da persistência.

Services concentram as regras que não podem falhar

Enviar moedas

O Service coordena a operação com controle transacional por **@Transactional**.

1. **Débito**
   Retira moedas do remetente.
2. **Crédito**
   Entrega moedas ao destinatário.
3. **Registro**
   Grava a transação.



Uma única operação

Se uma etapa falha, a operação inteira é desfeita.

A mesma camada também trata envio de e-mails e crédito semestral de moedas aos professores.

06 · PERSISTÊNCIA ORM

O ORM reduz o código entre a regra e o banco

O Service chama o DAO; as camadas superiores não precisam escrever SQL.

01

Service

Chama o DAO.

02

JpaRepository

Delega ao Hibernate.

03

Hibernate

Gera o SQL.

04

H2

Persiste em **backend/data/**.

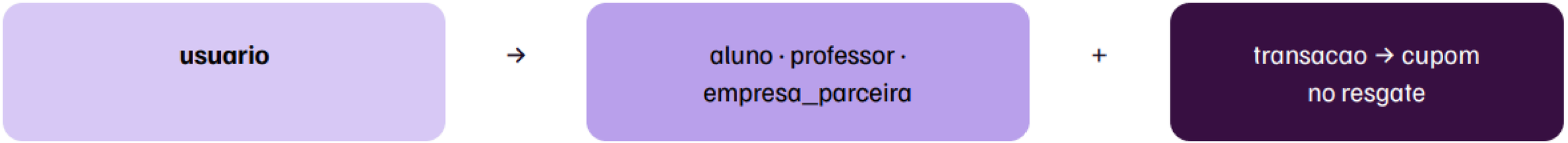
Menos acoplamento: sem SQL ou EntityManager nas camadas superiores.

Conversão: entidades viram DTOs ainda no Service.

Configuração: ddl-auto=update; open-in-view desativado.

Duas estratégias de herança resolvem problemas diferentes

Estratégia	Aplicação	Por que usar
JOINED	**Usuario** → Aluno, Professor, EmpresaParceira	Tabelas por classe, chave compartilhada; evita colunas nulas e centraliza login/e-mail.
SINGLE_TABLE	**Transacao** → Envio, Resgate	Uma tabela com discriminador **tipo**; simplifica o extrato.



DAOs entregam CRUD pronto e consultas específicas

7

interfaces

Estendem **JpaRepository**.

Query methods

existsByCpf, **findByLogin**, **findAllByOrderByNomeAsc**

Consultas específicas

@Query no TransacaoDAO cobre extratos, resgates, validação de cupom e bloqueios.

Trocar H2 por PostgreSQL ou MySQL exige driver e configuração, não novos DAOs.

Pacotes organizam responsabilidades do back-end

controller: API REST

service: regras

dao: persistência

model: entidades JPA

dto: entrada e saída

config: autenticação, CORS, SPA, uploads e dados iniciais

exception: tratamento de erros